Makefiles

```
# exemplo de makefile

pmedia: media.o main.o
        gcc media.o main.o -o pmedia

media.o: media.c
        gcc -c media.c -o media.o

main.o: main.c
        gcc -c main.c -o main.o

run: pmedia
        ./pmedia

clean:
        rm media.o main.o pmedia
```

Programação Assembly

1. Operandos das Instruções
As instruções podem ter 4 tipos de operandos: constantes, registos, endereços e valores implícitos

Constantes:
As constantes são precedidas por um '$'. Exemplos: $10, $n. Este último exemplo representa o endereço correspondente à etiqueta n.

Registos:
Um operando do tipo registo inicia sempre por '%'. Podem-se usar de seguida os 8 registos de 32 bits de uso geral (eax, ebx, ecx, edx, ebp, edi, esi e esp).
O registo esp esta reservado para apontador para o topo da stack. Pode-se usar, da mesma forma, os resgistos de 16 e 8 bits (ver lista de registos).

Endereços:
Um operando do tipo endereço refere-se a um valor num local de memória. O exemplo mais comum é a etiqueta de uma variável ou instrução.

O assemblador pode avaliar expressões aritméticas simples relativas a endereços.
Exemplos:

Endereço	**Refere-se a:**
length	valor na etiqueta **length**
length+4	valor 4 bytes depois de **length**
length-4	valor 4 bytes antes de **length**
(%ebp)	valor no endereço contido no registo **ebp**
4(%ebp)	valor 4 bytes depois do endereço contido em **ebp**
memloc(%eax)	valor em **memloc+eax**
(%ebp,%eax,4)	valor em **ebp+4*eax**
memloc(%ebp, %eax, 4)	valor em **memloc+ebp+4*eax**

Implícitos:
Algumas instruções têm operandos implícitos, como por exemplo o idivl que usa os registos eax e edx.

2. Alocação de Memória

Exemplos de alocação de memória:

```
            .data
v32:        .long 0                 # 32 bits, valor inicial 0
v16:        .word 0xffff            # 16 bits, todos a 1
v8:         .byte '-'               # 8 bits, inicializado com o código ascii de '-'
vs:         .ascii “input”          # string com 5 bytes
vs0:        .asciz “input”          # string com 6 bytes, o último é o zero.
            .align 4                # alinhar num endereço múltiplo de 32
pilha:      .skip 1024              # “reservar” 1024 bytes
```

3. Exemplo de Programa Totalmente em Assembly

```
            .text                   # zona de código
            .global main
main:
            pushl $n                # fazer push do segundo argumento do scanf, o endereço de n
            pushl $sfmt             # fazer push do primeiro argumento do scanf, o endereço de sfmt
            call scanf              # invocar scanf(“%d”, &n)
            addl $8, %esp           # fazer o pop dos dois argumentos anteriores
            addl $1, n              # adicionar 1 ao valor contido no endereço n
            pushl n                 # fazer push do segundo argumento do printf, o endereço de n
            push $fmt               # fazer push do primeiro argumento do printf, o endereço de fmt
            call printf             # invocar printf(“%d\n”, n)
            addl $8, %esp           # fazer o pop dos dois argumentos anteriores
            pushl $0                # fazer o push do primeiro argumento do exit, o valor 0
            call exit               # invocar o exit(0) para terminar o processo/programa actual

            .data                   # zona de dados
n:          .long 0                 # variável n, do tipo long, inicializada a 0
fmt:        .asciz “%d\n”           # sequência de caracteres ASCII terminada por 0. Inicializada com “%d\n”
sfmt:       .asciz “%d”
```

Resumo de Intel 32 / Linux

1. Registos

32 bits	16 bits menos signif.	2º byte menos signif.	8 bits menos signif.
eax	ax	ah	al
ecx	cx	ch	cl
edx	dx	dh	dl
ebx	bx	bh	bl
esi	si	---	---
edi	di	---	---
esp	sp	---	---
ebp	bp	---	---

2. Flags

D: Direction	1= sentido de processamento das operações de strings do endereço maior para o menor
I: Interrupt	Se as interrupções podem ocorrer. 1=interrupções activas.
T: Trap	Modo passo-a-passo para depuração
C: Carry	O Resultado da operação sem sinal é muito grande ou inferior a zero. 1=excesso
O: Overflow	O resultado da operação com sinal é muito grande ou muito pequena. 1=overflow/underflow
S: Sign	Sinal do resultado. Apenas para inteiros. 1=negativo / 0=positivo
Z: Zero	O resultado da operação é zero. 1=zero
A: Aux. Carry	Similar ao Carry mas apenas para o nibble menos significativo
P: Parity	1=O resultado tem um número para de bits a 1.

3. Instruções

Notas:

Formato geral das instruções: **instr[b|w|l]** *origem*, *destino*

O **b**, **w** e **l** no final do nome da instrução indicam o tamanho dos operandos (b=byte; w=word; l=long)

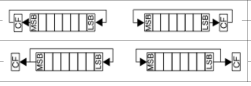
As letras nos operandos indicam o tipo dos operandos (r=register; a=address; c=constant)

O número 32 nos operandos indica o tamanho do operando. Se a instrução termina em **l** os operandos serão normalmente de 32 bits.

Movimentação

				Flags afectadas								
Nome	**Comentário**	**Código**	**Operação**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
movl	mover (Copiar)	movl rac32, ra32	ra32 := rac32									
pushl	push	pushl rac32	dec esp; [esp]:=rac32									
popl	pop	popl ra32	ra32 := [esp]; inc esp									

Aritméticas

				Flags afectadas								
Nome	**Comentário**	**Código**	**Operação**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
addl	adicionar	addl rac32, ra32	ra32 := ra32+rac32	?				?	?	?	?	?
adcl	adicionar com carry	adcl rac32, ra32	ra32 := ra32 + rac32 + CF	?				?	?	?	?	?
subl	subtrair	subl rac32, ra32	ra32 := ra32-rac32	?				?	?	?	?	?
sbbl	subtrair com carry	sbbl src, dst	ra32 := ra32 – (rac32+CF)	?				?	?	?	?	?
imull	multiplicar	Imull rac32, r32	r32 = r32*rac32	?				?	?	?	?	?
imull	multiplicar eax	imull r32	edx:eax = r32*eax	?				?	?	?	?	?
idivl	dividir	idivl ra32	eax = edx:eax/ra32, edx = resto	?				?	?	?	?	?
incl	incrementar	incl ra32	ra32 = ra32+1	?				?	?	?	?	
decl	decrementar	decl ra32	ra32 = ra32-1	?				?	?	?	?	
cmpl	comparar	cmpl $rac32_1$, $rac32_2$	comparar calculando $rac32_2$- $rac32_1$	?				?	?	?	?	?
sall	deslocar bits esquerda	sall cº, ra32		?				?	?	?	?	?
sarl	deslocar bits direita	sarl cº, ra32		?				?	?	?	?	?
rcll	rodar esquerda com carry	rcll cº, ra32		?								?
rcrl	rodar direita com carry	rcrl cº, ra32		?								?
roll	rodar esquerda	roll cº, ra32		?								?
rorl	rodar direita	rorl cº, ra32		?								?

º Em vez de uma constante entre 0 e 31 podemos ter o registo cl.

Lógicas

				Flags afectadas								
Nome	**Comentário**	**Código**	**Operação**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
negl	negar (complemento para 2)	negl ra32	ra32 = -ra32	?				?	?	?	?	?
notl	inverter bit a bit	notl ra32	ra32 = ~ra32, bitwise, false = 0									
andl	“e” lógico	andl rac32, ra32	ra32 = ra32 & rac32, bitwise					?	?	?	?	
orl	“ou” lógico	orl rac32, ra32	ra32 = ra32 I rac32, bitwise					?	?	?	?	
xorl	“ou” lógico exclusivo	xorl rac32, ra32	ra32 = ra32 ^ rac32, bitwise					?	?	?	?	
shll	deslocar bits esquerda	shll cº, ra32		?				?	?	?	?	?
shrl	deslocar bits direita	shrl cº, ra32		?				?	?	?	?	?

Saltos

				Flags afectadas								
Nome	**Comentário**	**Código**	**Operação**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
jmp	salto incondicional	jmp ra32										
jg	salto se maior	jg ra32										
jl	salto se menor	jl ra32										
je	salto se igual	je ra32										
jge	salto se maior ou igual	jge ra32										
jle	salto se menor ou igual	jle ra32										
jne	salto se diferente	jne ra32										
jc	saltar se carry=1	jc ra32										
jnc	saltar se carry=0	jnc ra32										
call	chamar rotina/função	call ra32										
ret	retornar de rotina/função	ret										

Diversas

				Flags afectadas								
Nome	**Comentário**	**Código**	**Operação**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
leal	carregar endereço	leal a32, r32	r32 = localização de a32									
nop	nenhuma operação	nop										